

Resource Configuration in NOAA Operational Workflows: A Technical Comparison of CROW and Global-Workflow Approaches

with MPMD Runtime Integration Analysis

National Oceanic and Atmospheric Administration
Environmental Modeling Center

Technical Report EMC-2026-001

Drafted collaboratively by Terry McGuinness and Anthropic’s Claude Opus 4.1, using the EIB MCP-RAG platform for code archaeology and literature retrieval.

Disclaimer: Originally generated by Claude Opus 4.1 through careful and detailed prompting followed by re-formatting and carefully reviewed and edited. All content is the sole responsibility of the author.

January 2026

Abstract

This technical report provides a comprehensive analysis comparing resource configuration methodologies between the Community Research Operations Workflow (CROW) system (2016–2020) and the current production GLOBAL-WORKFLOW (2020–present). We examine architectural differences in how computational resources—CPU ranks, threads, memory, walltime, and MPI execution—are specified, validated, and deployed across heterogeneous HPC platforms including WCOSS2, Hera, Orion, Hercules, and Gaea.

Through detailed code archaeology, formal algorithm analysis, and integration with our companion MPMD Runtime Infrastructure study, we demonstrate the trade-offs between CROW’s declarative YAML-based Domain-Specific Language (DSL) and GLOBAL-WORKFLOW’s imperative shell-based configuration. Our analysis covers the complete resource specification lifecycle: from definition through validation to runtime execution.

We conclude with concrete recommendations for next-generation workflow infrastructure that synthesizes the strengths of both approaches while addressing emerging requirements for GPU orchestration, cloud portability, and machine learning workflow integration. This analysis serves both as a historical record of the CROW system and as a technical foundation for future architectural decisions.

Contents

1	Introduction	5
1.1	The Resource Configuration Challenge	5
1.2	Historical Context	5
1.3	Scope and Methodology	6
2	Resource Specification Paradigms	6
2.1	Architectural Overview	6
2.2	CROW: Declarative Resource Tables	7
2.2.1	The Type System	7
2.2.2	Resource Table Structure	8
2.2.3	EnKF Resource Tables	8
2.2.4	JobRequest Construction	9
2.2.5	Downstream Resource Tables	9
2.3	Global-Workflow: Imperative Configuration	10
2.3.1	Configuration Hierarchy	10
2.3.2	Base Resource Configuration	10
2.3.3	Platform-Specific Overrides	12
2.3.4	Platform Environment Files	12
2.3.5	ecFlow Job Script Integration	13
3	MPI Execution Patterns	14
3.1	The SPMD/MPMD Dichotomy	14
3.2	CROW: Abstract Parallelism Layer	14
3.2.1	Platform Definition	14
3.2.2	WCROSS Configuration (Historical)	15
3.3	Global-Workflow: MPMD Wrapper System	16
3.3.1	MPMD Execution Architecture	16
3.3.2	Command File Generation	16
3.3.3	CFP (Coupled Framework Parallelism)	18
3.4	Launcher Abstraction Comparison	18
4	Comparative Analysis	18
4.1	Code Organization	18
4.2	Adding a New Resolution	19
4.2.1	CROW Approach	19
4.2.2	Global-Workflow Approach	19
4.3	Adding a New HPC Platform	20
4.3.1	CROW Approach	20
4.3.2	Global-Workflow Approach	21
4.4	Validation Capabilities	22
4.5	Error Handling Patterns	22
4.5.1	CROW: Fail-Fast	22
4.5.2	Global-Workflow: Defensive Defaults	22
4.6	Maintenance Metrics	23

5	Resource Tuple Semantics	23
5.1	CROW Tuple Format	23
5.2	Global-Workflow Variables	24
5.3	Semantic Comparison	24
5.4	Thread Configuration Semantics	24
6	Execution Flow Comparison	25
6.1	CROW Execution Path	25
6.2	Global-Workflow Execution Path	26
6.3	Execution Timeline Comparison	26
7	Memory and Thread Configuration	27
7.1	Memory Specification Models	27
7.1.1	CROW Memory Specification	27
7.1.2	Global-Workflow Memory	27
7.2	OpenMP Thread Management	28
7.2.1	Thread Calculation	28
7.2.2	CROW Thread Semantics	28
7.2.3	Global-Workflow Thread Management	29
7.3	Hybrid MPI+OpenMP Topology	30
8	Error Handling and Validation	30
8.1	CROW: Fail-Fast Philosophy	30
8.2	Global-Workflow: Defensive Programming	31
8.3	Error Detection Timing	32
8.4	Error Message Quality	32
9	Practical Implications	32
9.1	Developer Experience	32
9.1.1	Debugging Workflow	32
9.1.2	Onboarding Metrics	34
9.2	Maintenance Burden	34
9.2.1	Lines of Code Analysis	34
9.2.2	Change Frequency Analysis	34
9.3	Testing Implications	35
9.3.1	CROW Testing	35
9.3.2	Global-Workflow Testing	36
10	Comprehensive Feature Matrix	37
11	Recommendations	39
11.1	Architectural Recommendation: Hybrid Approach	39
11.2	Specific Recommendations	39
11.2.1	Recommendation 1: Standard Schema Language	39
11.2.2	Recommendation 2: Explicit Compilation Step	40
11.2.3	Recommendation 3: Preserve MPMD Infrastructure	40
11.2.4	Recommendation 4: Validation Pipeline	41
11.2.5	Recommendation 5: Future Infrastructure Requirements	42
11.3	Migration Strategy	42

11.4 Summary of Recommendations	43
12 Conclusion	43
12.1 Key Findings	43
12.2 Verdict: Context-Dependent Choice	44
12.3 Path Forward	44
12.4 Future Work	44
A CROW Custom Tag Reference	45
B Global-Workflow APRUN Variables	46

1 Introduction

The Global Forecast System (GFS) represents one of the most computationally demanding operational weather prediction systems in the world. A single forecast cycle involves atmospheric dynamics using the FV3 dynamical core, physics parameterizations across multiple schemes, data assimilation through GSI or 4DVar, ensemble generation (GEFS), post-processing through UPP, and product generation for downstream consumers. Each component has distinct resource requirements that vary dramatically by model resolution (C48 through C1152) and HPC platform characteristics.

1.1 The Resource Configuration Challenge

Operational weather forecasting demands precise control over computational resources. Misconfigured jobs can result in:

- **Forecast delays:** Under-provisioned jobs miss operational deadlines
- **Resource waste:** Over-provisioned jobs consume scarce HPC allocation
- **Job failures:** Incorrect MPI topology causes communication errors
- **Silent degradation:** Wrong thread counts produce correct but slow results

The challenge is compounded by the heterogeneous nature of NOAA’s HPC fleet:

Table 1: NOAA HPC Platform Characteristics (2025)

Platform	Scheduler	MPI	Cores/Node	Mem (GB)	Interconnect
WCSS2	PBS Pro	Cray MPICH	128	512	Slingshot
Hera	Slurm	Intel MPI	40	384	InfiniBand HDR
Orion	Slurm	Intel MPI	40	192	InfiniBand HDR
Hercules	Slurm	Intel MPI	80	512	InfiniBand NDR
Gaea C5/C6	Slurm	Cray MPICH	128	512	Slingshot

1.2 Historical Context

This report examines two fundamentally different approaches developed to address this challenge:

1. **CROW (2016–2020):** A declarative Domain-Specific Language (DSL) built on YAML with custom type extensions, designed to provide centralized, type-safe resource specifications with compile-time validation.
2. **GLOBAL-WORKFLOW (2020–present):** An imperative approach using shell scripts with conditional logic, chosen for its transparency, debuggability, and compatibility with existing operational practices.

Figure 1 illustrates the evolution of resource configuration approaches at NOAA/EMC.

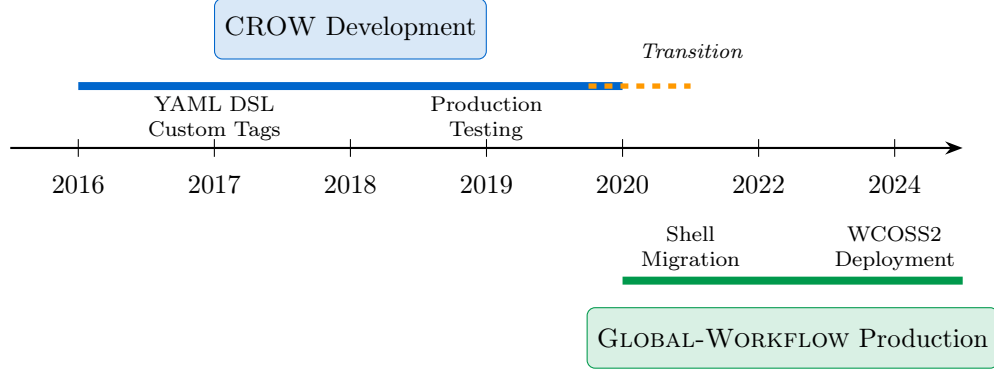


Figure 1: Timeline of resource configuration approaches at NOAA/EMC

1.3 Scope and Methodology

This analysis draws from:

- Primary source code from NOAA-EMC/global-workflow (current)
- Archived CROW codebase (workflow/ directory, pre-2020)
- Platform configuration files for all production HPC systems
- Our companion MPMD Runtime Infrastructure analysis (2025)
- Operational experience from forecast cycle deployments

We employ a structured comparison methodology, examining each system across four dimensions:

1. **Specification:** How resources are defined
2. **Validation:** How errors are detected
3. **Execution:** How specifications become runtime behavior
4. **Maintenance:** How changes propagate through the system

2 Resource Specification Paradigms

2.1 Architectural Overview

Figure 2 contrasts the data flow in both systems from resource definition to job submission.

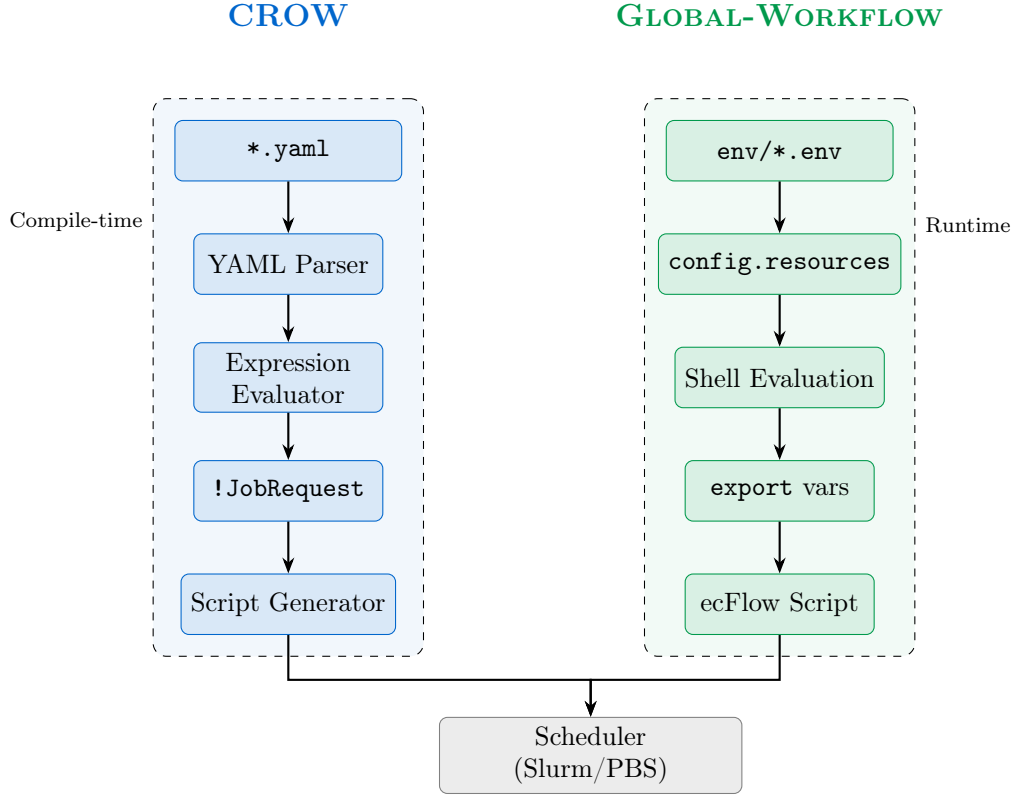


Figure 2: Architectural comparison: CROW (left) performs resolution at YAML parse time; GLOBAL-WORKFLOW (right) resolves at shell evaluation time

2.2 CROW: Declarative Resource Tables

CROW centralized all resource definitions in `workflow/defaults/default_resources.yaml` using resolution-indexed lookup tables with custom YAML type extensions.

2.2.1 The Type System

CROW extended standard YAML with custom tags forming a Domain-Specific Language (DSL):

Table 2: CROW Custom YAML Tags

Tag	Type	Purpose
<code>!Select</code>	Pattern Match	Conditional selection based on configuration value
<code>!calc</code>	Expression	Deferred Python expression evaluation
<code>!icalc</code>	Immediate Calc	Immediate expression evaluation during parse
<code>!JobRequest</code>	Resource Spec	Structured job resource specification
<code>!timedelta</code>	Duration	Time duration with HH:MM:SS parsing
<code>!error</code>	Exception	Explicit failure for undefined cases
<code>!Platform</code>	Platform Def	HPC platform configuration container
<code>!FirstTrue</code>	Conditional	First matching condition selection

2.2.2 Resource Table Structure

The core GFS resource table demonstrates CROW's resolution-indexed approach:

Listing 1: CROW GFS Resource Table (from `default_resources.yaml`)

```

1 gfs_resource_table: !Select
2   select: !icalc doc.fv3_gfs_settings.CASE
3   otherwise: !error Unknown GFS case
4   cases:
5     C96: &gfs_resource_C96
6       prep: [ 12, 12, !timedelta "00:15:00", null, null ]
7       anal: [ 144, 6, !timedelta "01:30:00", max, null ]
8       fcst: [ 168, 12, !timedelta "00:30:00", 2, null ]
9       post: [ 84, 12, !timedelta "02:00:00", 1, null ]
10      vrfy: [ 1, 1, !timedelta "03:00:00", null, null ]
11     C384: &gfs_resource_C384
12       prep: [ 4, 4, !timedelta "00:15:00", max, null ]
13       anal: [ 360, 6, !timedelta "01:30:00", max, null ]
14       fcst: [ 384, 6, !timedelta "02:30:00", 4, null ]
15       post: [ 240, 8, !timedelta "06:00:00", 1, null ]
16       vrfy: [ 3, 1, !timedelta "05:00:00", null, null ]
17     C768: &gfs_resource_C768
18       prep: [ 4, 4, !timedelta "00:15:00", max, null ]
19       anal: [ 480, 6, !timedelta "02:30:00", max, null ]
20       fcst: [ 768, 6, !timedelta "05:00:00", 4, null ]
21       post: [ 480, 8, !timedelta "08:00:00", 1, null ]
22       vrfy: [ 5, 1, !timedelta "06:00:00", null, null ]

```

The tuple format follows a consistent schema:

$$\text{tuple} = [\text{ranks}, \text{ppn}, \text{walltime}, \text{threads}, \text{memory}] \quad (1)$$

Where:

- **ranks**: Total MPI processes
- **ppn**: Processes per node (determines node count: $\lceil \text{ranks}/\text{ppn} \rceil$)
- **walltime**: Maximum job duration (!timedelta parsed)
- **threads**: OpenMP threads per rank (max = all available, null = platform default)
- **memory**: Per-node memory limit (null = platform default)

2.2.3 EnKF Resource Tables

The ensemble Kalman filter required separate resource tables due to its different scaling characteristics:

Listing 2: CROW EnKF Resource Table

```

1 enkf_resource_table: !Select
2   select: !icalc doc.fv3_gfs_settings.CASE
3   otherwise: !error Unknown EnKF case
4   cases:
5     C384: &enkf_C384
6       eobs: [ 360, 6, !timedelta "00:30:00", null, null ]
7       eomg: [ 360, 6, !timedelta "01:00:00", null, null ]
8       eupd: [ 960, 6, !timedelta "00:35:00", 4, null ]
9       ecen: [ 80, 8, !timedelta "00:30:00", 2, null ]
10      esfc: [ 78, 6, !timedelta "00:10:00", null, null ]
11      efcs: [ 240, 6, !timedelta "02:30:00", 2, null ]

```



```

12      C768: &enkf_C768
13      eobs:  [ 480, 6, !timedelta "00:45:00", null, null ]
14      eomg:  [ 480, 6, !timedelta "01:30:00", null, null ]
15      eupd:  [1200, 6, !timedelta "01:00:00", 4, null ]
16      ecen:  [ 120, 8, !timedelta "00:45:00", 2, null ]
17      esfc:  [ 156, 6, !timedelta "00:15:00", null, null ]
18      efcs:  [ 384, 6, !timedelta "04:00:00", 2, null ]

```

2.2.4 JobRequest Construction

Resources were consumed through `!JobRequest` objects that referenced the lookup tables:

Listing 3: CROW JobRequest Pattern

```

1  # Default JobRequest consuming gfs_resource_table
2  run_gdasfcst: !JobRequest
3    - batch_memory: !icalc >-
4      doc.gfs_resource_table.fcst[4]
5    - mpi_ranks: !icalc >-
6      doc.gfs_resource_table.fcst[0]
7    - OMP_NUM_THREADS: !icalc >-
8      doc.gfs_resource_table.fcst[3]
9    - walltime: !icalc >-
10      tools.to_timedelta(doc.gfs_resource_table.fcst[2])
11    - OMP_STACKSIZE: !icalc tools.OMP_STACKSIZE_default
12    - max_ppn: !icalc >-
13      doc.gfs_resource_table.fcst[1]
14
15  # Coupled model with different resources
16  run_coupled_fcst: !JobRequest
17    - batch_memory: !icalc >-
18      doc.coupled_resource_table.fcst[4]
19    - mpi_ranks: !icalc >-
20      doc.coupled_resource_table.fcst[0]
21    - walltime: !icalc >-
22      tools.to_timedelta(doc.coupled_resource_table.fcst[2])
23    - OMP_NUM_THREADS: !icalc >-
24      doc.coupled_resource_table.fcst[3] or 1
25    - max_ppn: !icalc >-
26      doc.coupled_resource_table.fcst[1]

```

2.2.5 Downstream Resource Tables

Post-processing and product generation jobs had different resource profiles:

Listing 4: CROW Downstream Resources

```

1 downstream_resource_table: !Select
2   select: !icalc doc.fv3_gfs_settings.CASE
3   otherwise: !error Unknown downstream case
4   cases:
5     C96:
6       awips_g1: [ 24, 24, !timedelta "00:05:00", 1, !calc memory.per_node ]
7       awips_g2: [ 24, 24, !timedelta "00:05:00", 1, !calc memory.per_node ]
8       gempak: [ 24, 24, !timedelta "00:05:00", 1, !calc memory.per_node ]
9       bufrsnd: [ 80, 8, !timedelta "00:20:00", 1, !calc memory.per_node ]
10      postsnd: [ 12, 12, !timedelta "00:30:00", 1, !calc memory.per_node ]
11    C768:
12      awips_g1: [ 24, 24, !timedelta "00:10:00", 1, !calc memory.per_node ]
13      awips_g2: [ 24, 24, !timedelta "00:10:00", 1, !calc memory.per_node ]
14      gempak: [ 24, 24, !timedelta "00:10:00", 1, !calc memory.per_node ]
15      bufrsnd: [ 80, 8, !timedelta "00:45:00", 1, !calc memory.per_node ]
16      postsnd: [ 24, 12, !timedelta "01:00:00", 1, !calc memory.per_node ]

```

2.3 Global-Workflow: Imperative Configuration

The current GLOBAL-WORKFLOW system employs a three-level configuration hierarchy using shell scripts with conditional logic.

2.3.1 Configuration Hierarchy

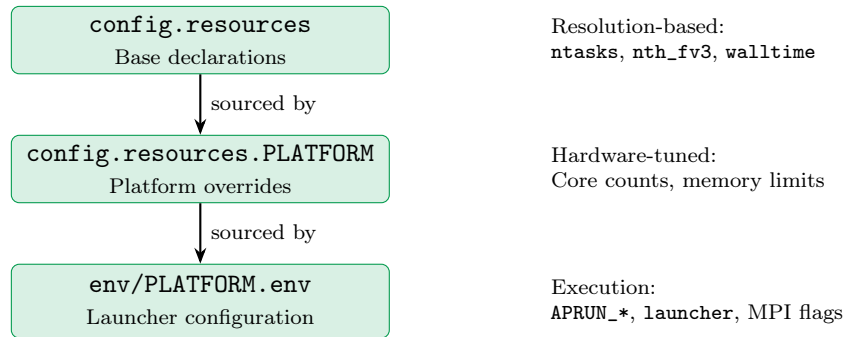


Figure 3: Three-level GLOBAL-WORKFLOW configuration hierarchy

2.3.2 Base Resource Configuration

The primary resource file uses nested case statements for resolution and step:

Listing 5: GLOBAL-WORKFLOW Base Resource Configuration (`config.resources`)

```

1 #!/bin/bash
2
3 # Forecast resources by resolution
4 case "${step}" in
5   "fcst"|"efcs")
6     export layout_x="--layout_x"
7     export layout_y="--layout_y"
8
9   case "${CASE}" in
10     "C48")
11       export layout_x=1
12       export layout_y=1
13       export ntasks_fv3=6

```

```

14     export ntasks_quilt=6
15     export nth_fv3=2
16     export nth_quilt=1
17     export walltime="00:30:00"
18     ;;
19     "C96")
20     export layout_x=4
21     export layout_y=4
22     export ntasks_fv3=96
23     export ntasks_quilt=24
24     export nth_fv3=2
25     export nth_quilt=1
26     export walltime="01:00:00"
27     ;;
28     "C384")
29     export layout_x=8
30     export layout_y=8
31     export ntasks_fv3=384
32     export ntasks_quilt=72
33     export nth_fv3=4
34     export nth_quilt=1
35     export walltime="02:30:00"
36     ;;
37     "C768")
38     export layout_x=12
39     export layout_y=12
40     export ntasks_fv3=864
41     export ntasks_quilt=144
42     export nth_fv3=4
43     export nth_quilt=1
44     export walltime="05:00:00"
45     ;;
46     "C1152")
47     export layout_x=16
48     export layout_y=16
49     export ntasks_fv3=1536
50     export ntasks_quilt=192
51     export nth_fv3=4
52     export nth_quilt=1
53     export walltime="08:00:00"
54     ;;
55     esac
56
57     # Total tasks = FV3 + Quilt
58     export ntasks=$((ntasks_fv3 + ntasks_quilt))
59     ;;
60
61     "anal"|"analc"|"analdiag")
62     case "${CASE}" in
63         "C48") export ntasks=60 ; export nth=4 ;;
64         "C96") export ntasks=120 ; export nth=8 ;;
65         "C384") export ntasks=480 ; export nth=8 ;;
66         "C768") export ntasks=960 ; export nth=8 ;;
67     esac
68     export walltime="{walltime_anal:-02:00:00}"
69     ;;
70
71     "post"|"postanl")
72     case "${CASE}" in
73         "C48") export ntasks=24 ; export nth=1 ;;
74         "C96") export ntasks=84 ; export nth=1 ;;
75         "C384") export ntasks=240 ; export nth=1 ;;
76         "C768") export ntasks=480 ; export nth=1 ;;
77     esac
78     ;;
79     esac
80
81     # Memory calculation

```

```
82 export memory="${memory:-${(ntasks * 4)}}GB"
```

2.3.3 Platform-Specific Overrides

Each platform can override base resources for hardware optimization:

Listing 6: Platform Override (`config.resources.HERA`)

```
1 #!/bin/bash
2 # HERA-specific resource overrides
3
4 source "${HOME}gfs}/parm/config/config.resources"
5
6 # Hera has 40 cores/node - adjust ppn
7 case "${step}" in
8   "fcst")
9     export ppn=20 # Leave room for I/O
10    export NTHREADS_FV3=${nth_fv3:-4}
11    ;;
12   "anal")
13     export ppn=10 # Heavy memory per rank
14    ;;
15   esac
16
17 # Memory constraints (Hera has 384GB/node)
18 export memory="180GB"
```

2.3.4 Platform Environment Files

The final layer configures MPI launchers and execution parameters:

Listing 7: HERA Platform Environment (`env/HERA.env`)

```
1 #!/bin/bash
2 # HERA - Slurm/Intel MPI Configuration
3
4 # System limits
5 ulimit -s unlimited
6 ulimit -a
7
8 # Launcher configuration
9 launcher="srun -l --export=ALL"
10 mpmd_opt="--multi-prog"
11
12 # MPI tuning for Intel MPI
13 export I_MPI_EXTRA_FILESYSTEM=on
14 export I_MPI_EXTRA_FILESYSTEM_FORCE=gafs
15 export I_MPI_ADJUST_ALLREDUCE=5
16
17 # Construct APRUN commands per component
18 case "${step}" in
19   "fcst"|"efcs")
20     export APRUN_UFS="${launcher} -n ${ntasks}"
21     export APRUN_FV3="${launcher} -n ${ntasks_fv3} --cpus-per-task=${nth_fv3}"
22     ;;
23   "anal"|"analc"|"analc")
24     export APRUN_GSI="${launcher} -n ${ntasks} --cpus-per-task=${nth}"
25     ;;
26   "post"|"postanl")
27     export APRUN_UPP="${launcher} -n ${ntasks}"
28     ;;
29   "prep"|"prepbufr")
```

```

30     export APRUN_PREP="${launcher} -n 4"
31     ;;
32 esac
33
34 # Generic commands
35 export APRUN_GAUSSIAN="${launcher} -n ${ntasks} --cpus-per-task=${nth:-1}"
36 export APRUN_CHGRES="${launcher} -n 6"
37 export APRUN_REGRID="${launcher} -n ${ntasks}"

```

Listing 8: WCOSS2 Platform Environment (env/WCOSS2.env)

```

1  #!/bin/bash
2  # WCOSS2 - PBS Pro/Cray MPICH Configuration
3
4  # System limits
5  ulimit -s unlimited
6  ulimit -a
7
8  # Cray-specific settings
9  export MPICH_MPIIO_HINTS_DISPLAY=1
10 export MALLOC_MMAP_MAX=0
11 export MALLOC_TRIM_THRESHOLD_=134217728
12
13 # PBS Pro launcher
14 launcher="mpiexec -l"
15 launcher_CFP="mpiexec -np ${nprocs} --cpu-bind verbose,core cfp"
16 mpmd_opt="--cpu-bind verbose,core cfp"
17
18 # Module loading for CFP
19 module load cfp/2.0.4 || true
20
21 # APRUN construction
22 case "${step}" in
23   "fcst"|"efcs")
24     export APRUN_UFS="${launcher} -np ${ntasks} --cpu-bind core"
25     export APRUN_FV3="${launcher} -np ${ntasks_fv3} --depth ${nth_fv3}"
26     ;;
27   "anal"|"analc"|"analc")
28     export APRUN_GSI="${launcher} -np ${ntasks} --depth ${nth}"
29     ;;
30   "post"|"postanl")
31     export APRUN_UPP="${launcher} -np ${ntasks}"
32     ;;
33   esac

```

2.3.5 ecFlow Job Script Integration

Resources flow into ecFlow job scripts through variable substitution:

Listing 9: ecFlow Job Script Header

```

1  #!/bin/bash
2  #PBS -N %JOBNAME%
3  #PBS -A %ACCOUNT%
4  #PBS -q %QUEUE%
5  #PBS -l walltime=%WALLTIME%
6  #PBS -l select=%NODES%:ncpus=%NCPUS%:mem=%MEMORY%
7  #PBS -l place=scatter
8  #PBS -j oe
9  #PBS -o %COM%/logs/%PDY%/%CYCINC%/%JOBNAME%.log
10
11 # Or for Slurm:
12 #SBATCH --job-name=%JOBNAME%
13 #SBATCH --account=%ACCOUNT%

```

```

14 #SBATCH --partition=%QUEUE%
15 #SBATCH --time=%WALLTIME%
16 #SBATCH --nodes=%NODES%
17 #SBATCH --ntasks=%NTASKS%
18 #SBATCH --cpus-per-task=%NCPUS%
19 #SBATCH --mem=%MEMORY%
20 #SBATCH --output=%COM%/logs/%PDY%/%CYCINC%/%JOBNAME%.log
21
22 # Source configuration hierarchy
23 source "${HOMEgfs}/parm/config/config.base"
24 source "${HOMEgfs}/parm/config/config.resources"
25 source "${HOMEgfs}/env/${machine}.env"

```

3 MPI Execution Patterns

This section integrates findings from our companion MPMD Runtime Infrastructure analysis, examining how both systems translate resource specifications into actual MPI execution.

3.1 The SPMD/MPMD Dichotomy

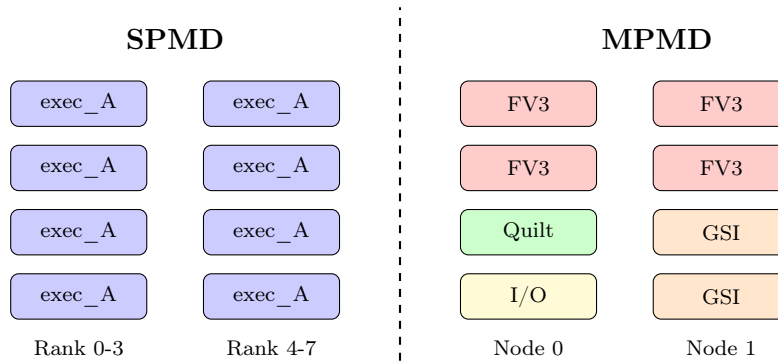


Figure 4: SPMD (single program) vs MPMD (multiple programs) execution models

3.2 CROW: Abstract Parallelism Layer

CROW generated scheduler-agnostic job specifications through platform configuration files that abstracted the execution model.

3.2.1 Platform Definition

Listing 10: CROW Hera Platform Configuration

```

1 hera: !Platform
2   name: hera
3   physical_cores_per_node: 40
4   hyperthreading_allowed: true
5   logical_cores_per_node: 80
6
7   scheduler_settings: !icalc |
8     tools.get_scheduler(
9       sched='Slurm',
10      machine='hera',
11      partition='hera')
12

```

```

13 partitions:
14     default: &default_partition
15     specification: !icalc |
16         tools.get_parallelism('HydraIMPI')
17     scheduler_settings:
18         partition: hera
19         qos: batch
20         account: !calc doc.platform.accounting.account
21
22     bigmem:
23     specification: !icalc |
24         tools.get_parallelism('HydraIMPI')
25     scheduler_settings:
26         partition: bigmem
27         qos: batch
28
29     service:
30     specification: !icalc |
31         tools.get_parallelism('LocalSerial')
32     scheduler_settings:
33         partition: service
34         qos: batch
35
36 parallelism: !calc |
37     tools.get_parallelism('HydraIMPI')
38
39 scheduler: !calc |
40     tools.get_scheduler('Slurm')
41
42 nodes: !calc |
43     tools.get_node_count(
44         doc.platform.physical_cores_per_node,
45         hyperthreading=True)

```

3.2.2 WCOSS Configuration (Historical)

Listing 11: CROW WCOSS Dell P3 Configuration

```

1 wcss_dell_p3: !Platform
2     name: wcss_dell_p3
3     physical_cores_per_node: 28
4     hyperthreading_allowed: false
5
6     scheduler_settings: !icalc |
7         tools.get_scheduler(
8             sched='LSF',
9             machine='dell_p3')
10
11     use_task_geometry: false # CFP mode
12
13     partitions:
14     default:
15     specification: !icalc |
16         tools.get_parallelism('LSFpoe')
17     scheduler_settings:
18         partition: !FirstTrue
19         - when: !calc tools.can_write('/gpfs/hps2/ptmp/')
20             do: dev
21         - otherwise: devmax
22         account: !calc doc.platform.accounting.account
23
24     transfer:
25     specification: !icalc |
26         tools.get_parallelism('LocalSerial')
27     scheduler_settings:

```

```

28     partition: transfer
29
30     # CFP module for MPMD
31     rocoto_load_modules: |
32         module load cfp
33         module load ics

```

Algorithm 1 describes how CROW resolved resource specifications:

Algorithm 1 CROW Resource Resolution

Require: YAML configuration tree D , platform P , job J

Ensure: Scheduler-specific job script

```

1: CASE  $\leftarrow D.fv3\_gfs\_settings.CASE$  {Get resolution}
2: table  $\leftarrow SELECT(D.gfs\_resource\_table, CASE)$ 
3: if table = !error then
4:     raise ConfigurationError
5: end if
6: tuple  $\leftarrow table[J.step]$  {[ranks, ppn, wall, threads, mem]}
7: req  $\leftarrow BuildJobRequest(tuple)$ 
8: sched  $\leftarrow P.scheduler\_settings.scheduler\_name$ 
9: script  $\leftarrow GenerateScript(req, sched, P)$ 
10: return script

```

3.3 Global-Workflow: MPMD Wrapper System

The current GLOBAL-WORKFLOW system uses `ush/run_mpmd.sh` as a unified interface for MPMD execution across schedulers.

3.3.1 MPMD Execution Architecture

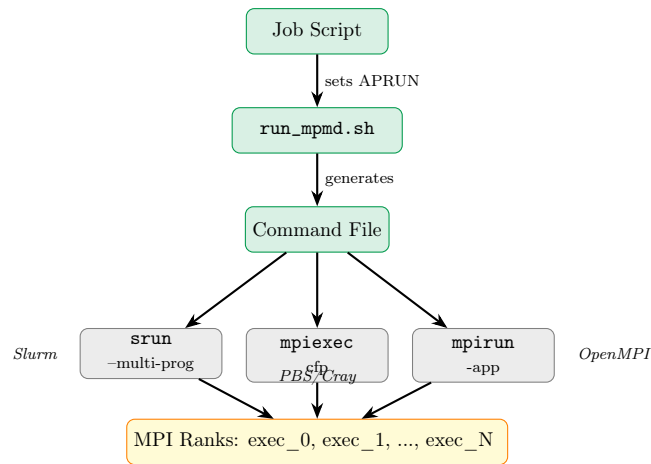


Figure 5: GLOBAL-WORKFLOW MPMD execution flow through `run_mpmd.sh`

3.3.2 Command File Generation

Listing 12: `run_mpmd.sh` Core Functions

```

1  #!/bin/bash

```



```

2 # run_mpmc.sh - Unified MPMD execution wrapper
3
4 # Generate MPMD command file
5 # Arguments: pairs of (nprocs, command)
6 # Output: cmdfile with one line per MPI rank
7 create_mpmc_cmdfile() {
8     local cmdfile=$1
9     shift
10
11     > "${cmdfile}" # Truncate
12     local rank=0
13
14     while (( $# >= 2 )); do
15         local nprocs=$1
16         local cmd=$2
17         shift 2
18
19         for (( i=0; i<nprocs; i++ )); do
20             if [[ "${SCHEDULER}" == "slurm" ]]; then
21                 # Slurm multi-prog format: rank command
22                 echo "${rank} ${cmd}" >> "${cmdfile}"
23             else
24                 # PBS/cfp format: command only
25                 echo "${cmd}" >> "${cmdfile}"
26             fi
27             ((rank++))
28         done
29     done
30
31     echo "Generated ${rank} rank entries in ${cmdfile}" >&2
32 }
33
34 # Execute MPMD job
35 run_mpmc() {
36     local cmdfile=$1
37     local total_ranks=$2
38
39     case "${launcher_type:-srun}" in
40         "srun")
41             # Slurm: srun with --multi-prog
42             ${APRUN_MPMc:-srun} \
43                 -n "${total_ranks}" \
44                 --multi-prog "${cmdfile}"
45             ;;
46         "mpiexec"|"mpiexec_mpt")
47             # PBS/Cray: mpiexec with cfp
48             module load cfp 2>/dev/null || true
49             ${APRUN_MPMc:-mpiexec} \
50                 -np "${total_ranks}" \
51                 --cpu-bind verbose,core \
52                 cfp "${cmdfile}"
53             ;;
54         "mpirun")
55             # OpenMPI: mpirun with -app
56             ${APRUN_MPMc:-mpirun} \
57                 -np "${total_ranks}" \
58                 -app "${cmdfile}"
59             ;;
60         *)
61             echo "ERROR: Unknown launcher_type: ${launcher_type}" >&2
62             return 1
63             ;;
64     esac
65 }

```

3.3.3 CFP (Coupled Framework Parallelism)

The CFP module enables MPMD execution on WCOSS2:

Listing 13: CFP Command File Format

```

1 # cmdfile.mpmc - One command per MPI rank
2 # Ranks 0-95: FV3 atmospheric model
3 /path/to/ufs_weather_model
4 /path/to/ufs_weather_model
5 ... (96 lines)
6 # Ranks 96-119: Quilt/Write component
7 /path/to/ufs_weather_model --write
8 /path/to/ufs_weather_model --write
9 ... (24 lines)
10 # Ranks 120-123: Post-processor
11 /path/to/ncep_post
12 /path/to/ncep_post
13 /path/to/ncep_post
14 /path/to/ncep_post

```

3.4 Launcher Abstraction Comparison

Both systems abstract the launcher, but at different levels:

Table 3: Launcher Abstraction Comparison

Aspect	CROW	GLOBAL-WORKFLOW
Abstraction level	YAML configuration	Shell variables
Resolution time	YAML parse	Runtime
Slurm command	Generated by !Platform	<code>\${launcher} = "srun -l"</code>
PBS command	Generated by !Platform	<code>\${launcher} = "mpiexec -l"</code>
MPMD syntax	Generated from !JobRequest	Manual cmdfile via <code>run_mpmc.sh</code>
Thread binding	<code>tools.get_parallelism()</code>	<code>-cpus-per-task, -depth</code>

4 Comparative Analysis

This section provides a structured comparison across the four dimensions identified in Section 1.

4.1 Code Organization

Table 4 compares where resource information resides in each system.

Table 4: File Organization Comparison

Component	CROW	GLOBAL-WORKFLOW
Resource definitions	workflow/defaults/default_resources.yaml	parm/config/config.resources
Platform config	workflow/platforms/*.yaml	env/*.env
Platform overrides	Inherited via YAML	parm/config/config.resources.*
Scheduler headers	Generated at parse time	ecf/scripts/*.ecf (embedded)
MPMD handling	Via !JobRequest composition	ush/run_mpmd.sh
MPI tuning	In platform YAML	In env/*.env

4.2 Adding a New Resolution

4.2.1 CROW Approach

Add one entry to each resource table:

Listing 14: Adding C1152 to CROW

```

1 gfs_resource_table: !Select
2   select: !icalc doc.fv3_gfs_settings.CASE
3   cases:
4     # ... existing cases ...
5     C1152: &gfs_resource_C1152
6     prep: [ 6, 6, !timedelta "00:20:00", max, null ]
7     anal: [ 720, 6, !timedelta "03:30:00", max, null ]
8     fcst: [ 1536, 6, !timedelta "08:00:00", 4, null ]
9     post: [ 720, 8, !timedelta "10:00:00", 1, null ]
10    vrfy: [ 8, 1, !timedelta "08:00:00", null, null]

```

Propagation: All jobs using `gfs_resource_table` automatically inherit the new resolution. The `!JobRequest` objects require no modification.

Lines changed: ~6 (one table entry)

4.2.2 Global-Workflow Approach

Add case branches to each step in `config.resources`:

Listing 15: Adding C1152 to GLOBAL-WORKFLOW

```

1 case "${step}" in
2   "fcst"|"efcs")
3     case "${CASE}" in
4       # ... existing cases ...
5       "C1152")
6         export layout_x=16
7         export layout_y=16
8         export ntasks_fv3=1536
9         export ntasks_quilt=192
10        export nth_fv3=4
11        export nth_quilt=1

```

```

12     export walltime="08:00:00"
13     ;;
14     esac
15     ;;
16     "anal"|"analc"|"analdiag")
17     case "${CASE}" in
18         # ... existing cases ...
19         "C1152")
20             export ntasks=720
21             export nth=8
22             export walltime="03:30:00"
23             ;;
24         esac
25         ;;
26     "post"|"postanl")
27     case "${CASE}" in
28         # ... existing cases ...
29         "C1152")
30             export ntasks=720
31             export nth=1
32             ;;
33         esac
34         ;;
35     # ... repeat for 15+ additional steps ...
36 esac

```

Propagation: Each step must be updated independently. Missing a step may cause runtime failures or incorrect resource allocation.

Lines changed: ~50-100 (depending on step count)

4.3 Adding a New HPC Platform

4.3.1 CROW Approach

Create a new platform YAML file:

Listing 16: New Platform in CROW

```

1 # workflow/platforms/aws_pcluster.yaml
2 aws_pcluster: !Platform
3   name: aws_pcluster
4   physical_cores_per_node: 96 # c6i.24xlarge
5   hyperthreading_allowed: true
6   logical_cores_per_node: 192
7
8   scheduler_settings: !icalc |
9     tools.get_scheduler(
10       sched='Slurm',
11       machine='aws_pcluster')
12
13   partitions:
14     compute:
15       specification: !icalc |
16         tools.get_parallelism('PMI2')
17       scheduler_settings:
18         partition: compute
19         account: !calc doc.platform.accounting.account
20
21     spot:
22       specification: !icalc |
23         tools.get_parallelism('PMI2')
24       scheduler_settings:
25         partition: spot
26
27   parallelism: !calc tools.get_parallelism('PMI2')

```

```
28 scheduler: !calc tools.get_scheduler('Slurm')
```

Files changed: 1 new file

4.3.2 Global-Workflow Approach

Create multiple files:

Listing 17: New Platform Environment (env/AWS.env)

```
1 #!/bin/bash
2 # AWS ParallelCluster - Slurm/PMI2 Configuration
3
4 ulimit -s unlimited
5
6 # PMI2 settings for AWS
7 export I_MPI_PMI_LIBRARY=/opt/slurm/lib/libpmi2.so
8 export I_MPI_FABRICS=shm:ofi
9 export FI_PROVIDER=efa
10
11 launcher="srun -l --export=ALL --mpi=pmi2"
12 mpmc_opt="--multi-prog"
13
14 # APRUN for each step (15+ definitions)
15 case "${step}" in
16   "fcst"|"efcs")
17     export APRUN_UFS="${launcher} -n ${ntasks}"
18     export APRUN_FV3="${launcher} -n ${ntasks_fv3}"
19     ;;
20   "anal"|"analc"|"analc")
21     export APRUN_GSI="${launcher} -n ${ntasks}"
22     ;;
23   # ... 10+ more cases ...
24 esac
```

Listing 18: Platform Resource Overrides (config.resources.AWS)

```
1 #!/bin/bash
2 # AWS-specific resource tuning
3
4 source "${HOME}gfs/parm/config/config.resources"
5
6 # AWS c6i.24xlarge: 96 cores, 192GB
7 case "${step}" in
8   "fcst")
9     export ppn=24 # 4 ranks * 4 threads = 96
10    ;;
11   "anal")
12     export ppn=12 # Heavy memory per rank
13    ;;
14   esac
15
16 export memory="180GB"
```

Additionally, update ecFlow templates with AWS-specific headers.

Files changed: 2-3 new files + template modifications

4.4 Validation Capabilities

Table 5: Validation Feature Comparison

Feature	CROW	GLOBAL-WORKFLOW
Schema validation	Yes (YAML)	No
Type checking	Yes (custom tags)	No
Undefined case detection	!error at parse	Runtime default
Expression syntax errors	Parse-time	Runtime
Missing variable detection	Parse-time	Runtime (or silent)
Cross-reference validation	Yes (doc refs)	No
Static analysis possible	Yes	Limited
IDE syntax support	Limited	Excellent
Syntax highlighting	YAML-based	Full Bash
Linting tools	Custom	ShellCheck

4.5 Error Handling Patterns

4.5.1 CROW: Fail-Fast

Listing 19: CROW Error Handling

```

1 gfs_resource_table: !Select
2   select: !icalc doc.settings.CASE
3   otherwise: !error |
4     Unknown resolution '{{doc.settings.CASE}}'.
5     Supported: C96, C192, C384, C768
6   cases:
7     C96: [...]
8     C384: [...]
9     C768: [...]
```

Behavior: Invalid configurations fail at YAML parse time, before job submission. Error messages include context.

4.5.2 Global-Workflow: Defensive Defaults

Listing 20: GLOBAL-WORKFLOW Default Handling

```

1 case "${CASE}" in
2   "C96") export ntasks=48 ;;
3   "C384") export ntasks=150 ;;
4   "C768") export ntasks=360 ;;
5   *)
6     echo "WARNING: Unknown CASE '${CASE}', using default" >&2
7     export ntasks=${ntasks:-24} # Fallback
8     ;;
9 esac
```

Behavior: Unknown configurations proceed with potentially incorrect defaults. May cause job failures or resource waste.

4.6 Maintenance Metrics

Table 6: Lines of Code for Resource Configuration

Component	CROW	GLOBAL-WORKFLOW	Notes
Core resource definitions	~350	~1,200	Resolution tables vs case statements
Platform config (per platform)	~60	~150	YAML vs shell + headers
JobRequest constructors	~200	N/A	Reusable patterns
MPMD wrapper	N/A	~150	<code>run_mpmc.sh</code>
Total (6 platforms)	~910	~2,250	

Table 7: Change Propagation Analysis

Change Type	CROW Files	GLOBAL-WORKFLOW Files
New resolution (C1152)	1	1-2
New platform	1	2-3
New step type	1-2	3-5
Walltime adjustment (global)	1	15+
Memory limit change	1	6+
Thread count change	1	15+

5 Resource Tuple Semantics

Both systems encode the same fundamental resource parameters, but with different representations and semantics.

5.1 CROW Tuple Format

CROW’s fixed-position tuples provide a compact, position-dependent encoding:

$$\text{tuple} = [\text{ranks}, \text{ppn}, \text{walltime}, \text{threads}, \text{memory}] \quad (2)$$

Table 8: CROW Tuple Field Semantics

Index	Field	Meaning	Special Values
0	ranks	Total MPI processes	Integer
1	ppn	Processes per node	Integer
2	walltime	HH:MM:SS duration	<code>!timedelta</code> object
3	threads	OpenMP threads/rank	<code>max</code> , <code>null</code> , Integer
4	memory	Per-node memory	<code>null</code> , String (e.g., "4G")

Node calculation:

$$\text{nodes} = \left\lceil \frac{\text{ranks}}{\text{ppn}} \right\rceil \quad (3)$$

Example: [240, 12, `!timedelta "01:30:00"`, 4, `null`]

- 240 MPI ranks total
- 12 processes per node $\Rightarrow$ 20 nodes
- 1:30:00 walltime limit
- 4 OpenMP threads per rank $\Rightarrow$ 48 threads/node
- Platform-default memory

5.2 Global-Workflow Variables

GLOBAL-WORKFLOW’s named variables provide explicit, self-documenting configuration:

Listing 21: GLOBAL-WORKFLOW Resource Variables

```

1 # Core resource specification
2 export ntasks=240          # Total MPI ranks
3 export ppn=12              # Processes per node
4 export walltime="01:30:00" # Walltime limit
5 export nth_fv3=4           # OMP threads for FV3
6 export memory="4GB"        # Per-node memory
7
8 # Derived values
9 export nodes=$(( ntasks + ppn - 1 ) / ppn ))
10 export threads_per_node=$(( ppn * nth_fv3 ))
11
12 # Component-specific (forecast)
13 export ntasks_fv3=216      # FV3 ranks
14 export ntasks_quilt=24     # Write component ranks
15 export ntasks=$(( ntasks_fv3 + ntasks_quilt ))
16
17 # Component-specific (analysis)
18 export nth=8               # GSI threads

```

5.3 Semantic Comparison

Table 9: Resource Parameter Mapping

Concept	CROW	GLOBAL-WORKFLOW
MPI ranks	tuple[0]	\$ntasks
Ranks per node	tuple[1]	\$ppn
Walltime	tuple[2] (!timedelta)	\$walltime (string)
OpenMP threads	tuple[3]	\$nth_fv3, \$nth
Memory limit	tuple[4]	\$memory
Node count	Calculated	Calculated or explicit
FV3-specific ranks	Via table lookup	\$ntasks_fv3
Quilt-specific ranks	Via table lookup	\$ntasks_quilt

5.4 Thread Configuration Semantics

Thread configuration demonstrates a key difference in expressiveness:

Listing 22: CROW Thread Semantics


```

1 # 'max' = use all available cores
2 fcst: [168, 12, !timedelta "00:30:00", max, null]
3
4 # Explicit thread count
5 anal: [144, 6, !timedelta "01:30:00", 4, null]
6
7 # 'null' = platform default (usually 1)
8 prep: [12, 12, !timedelta "00:15:00", null, null]

```

In GLOBAL-WORKFLOW, the **max** semantic must be explicitly calculated:

Listing 23: GLOBAL-WORKFLOW Thread Calculation

```

1 # Calculate 'max' threads
2 cores_per_node=40 # Platform-specific
3 nth_fv3=$((cores_per_node / ppn)) # max threads
4
5 # Or explicit
6 nth_fv3=4
7
8 # Default (null equivalent)
9 nth_fv3=${nth_fv3:-1}

```

6 Execution Flow Comparison

6.1 CROW Execution Path

CROW's execution follows a compilation model with distinct phases:

Algorithm 2 CROW Configuration Resolution

Require: Configuration directory $\mathcal{C}$, experiment E

Ensure: Scheduler-ready job scripts $\mathcal{S}$

```

1:  $D \leftarrow \text{LoadYAML}(\mathcal{C}/\text{defaults}/*.yaml)$  {Phase 1: Parse}
2:  $P \leftarrow \text{LoadYAML}(\mathcal{C}/\text{platforms}/E.\text{platform}.yaml)$ 
3: for all !icalc expression  $e$  in  $D$  do
4:    $D[e] \leftarrow \text{Evaluate}(e, \text{context}=D)$  {Phase 2: Immediate evaluation}
5: end for
6: for all !Select node  $s$  in  $D$  do
7:    $key \leftarrow \text{Evaluate}(s.\text{select})$ 
8:   if  $key \in s.\text{cases}$  then
9:      $D[s] \leftarrow s.\text{cases}[key]$ 
10:  else if  $s.\text{otherwise} = \text{!error}$  then
11:    raise ConfigurationError( $s.\text{otherwise}$ )
12:  else
13:     $D[s] \leftarrow s.\text{otherwise}$ 
14:  end if
15: end for{Phase 3: Pattern matching}
16: for all !JobRequest  $J$  in  $D$  do
17:    $R \leftarrow \text{BuildResources}(J, D)$ 
18:    $script \leftarrow \text{GenerateSchedulerScript}(R, P)$ 
19:    $\mathcal{S} \leftarrow \mathcal{S} \cup \{script\}$ 
20: end for{Phase 4: Script generation}
21: return  $\mathcal{S}$ 

```

Key Properties:

- All errors detected before job submission
- Scripts can be inspected before execution
- Configuration is immutable after compilation
- No runtime variable expansion in scheduler directives

6.2 Global-Workflow Execution Path

GLOBAL-WORKFLOW's execution follows an interpretation model with runtime resolution:

Algorithm 3 GLOBAL-WORKFLOW Configuration Resolution

Require: Job script J , platform P , step S , resolution R

Ensure: Executed job with allocated resources

```

1: source(config.base) {Load base configuration}
2: source(config.resources) {Load resource definitions}
3:  $resources \leftarrow \text{EvalCaseStatement}(\text{step}=S, \text{CASE}=R)$  {Shell case evaluation}
4: if  $resources = \text{empty}$  then
5:   warn("Unknown configuration")
6:    $resources \leftarrow \text{defaults}$ 
7: end if {Fallback on unknown}
8: export( $resources$ ) {Environment variables}
9: source(env/ $P$ .env) {Platform launcher config}
10:  $launcher \leftarrow \text{BuildAPRUN}(resources, P)$ 
11: SubmitToScheduler( $J, launcher$ ) {Variables expanded by scheduler}
```

Key Properties:

- Errors may occur at runtime
- Configuration can be modified at any point before submission
- Variables expanded by scheduler at submission time
- Debugging via standard shell tools (`set -x`, `echo`)

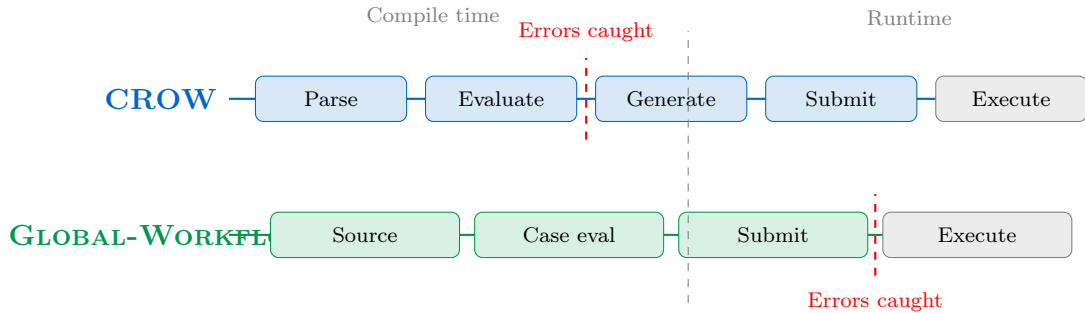
6.3 Execution Timeline Comparison

Figure 6: Error detection timing: CROW catches errors at compile time; GLOBAL-WORKFLOW at runtime

7 Memory and Thread Configuration

Memory and thread management represents one of the most complex aspects of HPC resource configuration, with significant differences between systems.

7.1 Memory Specification Models

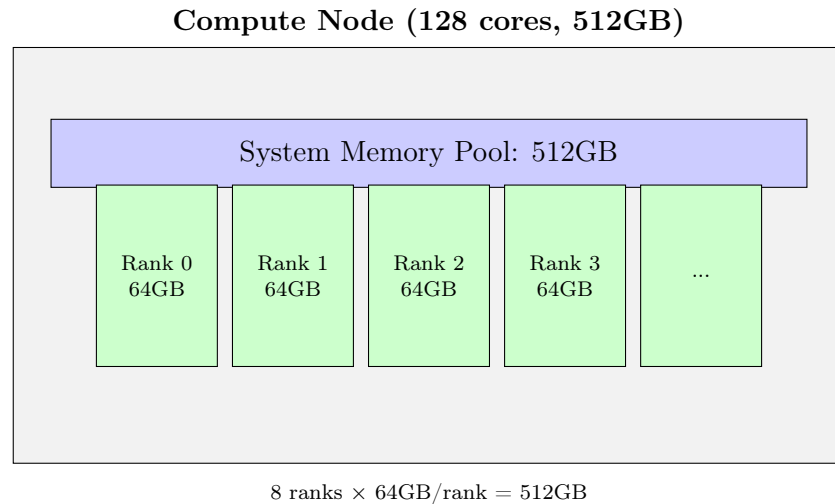


Figure 7: Per-rank vs per-node memory allocation

7.1.1 CROW Memory Specification

CROW supported multiple memory specification modes:

Listing 24: CROW Memory Options

```

1 # Per-node memory (most common)
2 gfs_forecast: !JobRequest
3   - batch_memory: '512G'           # Total per node
4   - mpi_ranks: 8
5   - max_ppn: 8
6
7 # Per-rank memory (for memory-bound jobs)
8 gsi_analysis: !JobRequest
9   - memory_per_rank: '64G'         # Per MPI rank
10  - mpi_ranks: 80
11  - max_ppn: 10
12
13 # Dynamic memory calculation
14 enkf_update: !JobRequest
15   - batch_memory: !calc >-
16     tools.calculate_memory(
17       ranks=doc.enkf_resource_table.eupd[0],
18       ppn=doc.enkf_resource_table.eupd[1],
19       base_per_rank='8G')

```

7.1.2 Global-Workflow Memory

GLOBAL-WORKFLOW's memory is typically specified per-node with platform-specific handling:

Listing 25: GLOBAL-WORKFLOW Memory Handling

```

1 # In config.resources
2 case "${step}" in
3   "fcst")
4     export memory="180GB"      # Per node
5     ;;
6   "anal")
7     export memory="384GB"      # Full node (GSI memory-bound)
8     ;;
9   "post")
10    export memory="96GB"       # Light memory needs
11    ;;
12 esac
13
14 # In ecFlow script (PBS)
15 #PBS -l select=${nodes}:ncpus=${ppn}:mem=${memory}
16
17 # In ecFlow script (Slurm)
18 #SBATCH --mem=${memory}
19 # or per-cpu:
20 #SBATCH --mem-per-cpu=${mem_per_cpu}

```

7.2 OpenMP Thread Management

7.2.1 Thread Calculation

Both systems must ensure thread counts respect node topology:

$$\text{threads_per_node} = \text{ppn} \times \text{threads_per_rank} \leq \text{cores_per_node} \quad (4)$$

Table 10: Thread Configuration Examples by Platform

Platform	Cores	ppn	threads	Total	Utilization
WCOSS2	128	8	16	128	100%
WCOSS2	128	16	4	64	50%
Hera	40	10	4	40	100%
Hera	40	20	1	20	50%
Hercules	80	20	4	80	100%

7.2.2 CROW Thread Semantics

Listing 26: CROW Thread Specification

```

1 # Thread specification in resource tuple
2 # Format: [ranks, ppn, walltime, threads, memory]
3
4 # Explicit thread count
5 fcst: [384, 6, !timedelta "02:30:00", 4, null]
6 # Result: 4 OMP threads per rank
7
8 # 'max' = fill remaining cores
9 anal: [360, 6, !timedelta "01:30:00", max, null]
10 # On Hera (40 cores): max = 40/6 = 6 threads
11 # On WCOSS2 (128 cores): max = 128/6 = 21 threads
12
13 # 'null' = platform default (typically 1)
14 prep: [12, 12, !timedelta "00:15:00", null, null]

```

```
15 # Result: 1 OMP thread per rank
```

The max semantic was implemented as:

Listing 27: CROW max Thread Calculation

```
1 OMP_NUM_THREADS: !icalc >-
2 doc.platform.physical_cores_per_node //
3 doc.gfs_resource_table.fcst[1]
4 if doc.gfs_resource_table.fcst[3] == 'max'
5 else doc.gfs_resource_table.fcst[3]
```

7.2.3 Global-Workflow Thread Management

Listing 28: GLOBAL-WORKFLOW Thread Configuration

```
1 # In config.resources
2 case "${step}" in
3   "fcst")
4     export nth_fv3=4          # FV3 threads
5     export nth_quilt=1        # Write threads
6     ;;
7   "anal")
8     # 'max' equivalent - calculate from platform
9     nth=$((nth:-$((cores_per_node / ppn)))
10    export nth
11    ;;
12 esac
13
14 # In env/HERA.env
15 export OMP_NUM_THREADS=${nth_fv3:-1}
16 export OMP_STACKSIZE=2048M
17 export OMP_PROC_BIND=spread
18 export OMP_PLACES=threads
19
20 # Thread binding for srun
21 export APRUN_FV3="${launcher} -n ${ntasks_fv3} \
22   --cpus-per-task=${nth_fv3}"
23
24 # Thread binding for mpiexec (WCSS2)
25 export APRUN_FV3="${launcher} -np ${ntasks_fv3} \
26   --depth ${nth_fv3} --cpu-bind depth"
```

7.3 Hybrid MPI+OpenMP Topology

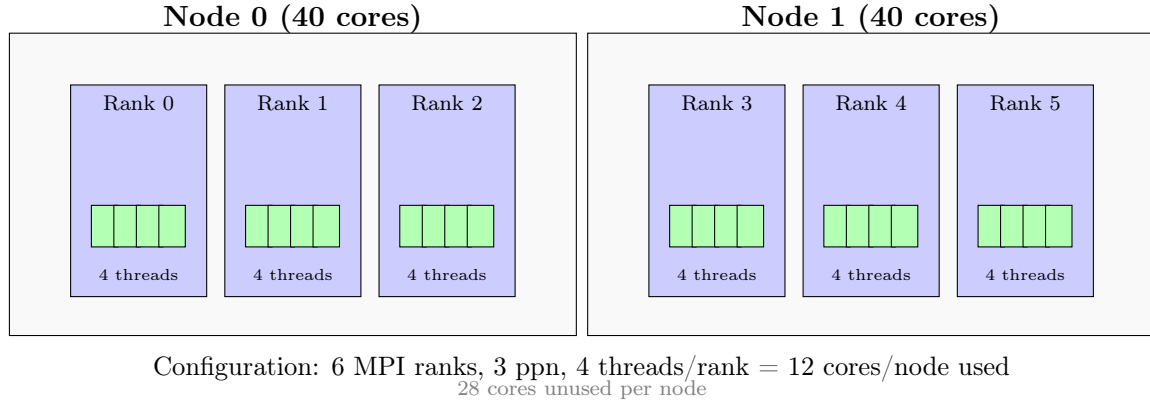


Figure 8: Hybrid MPI+OpenMP layout: 6 ranks across 2 nodes with 4 threads each

8 Error Handling and Validation

Error handling philosophy differs fundamentally between the two systems.

8.1 CROW: Fail-Fast Philosophy

Listing 29: CROW Comprehensive Error Handling

```

1 gfs_resource_table: !Select
2   select: !icalc doc.settings.CASE
3   otherwise: !error |
4     Configuration Error: Unknown model resolution.
5
6   Received: '{{doc.settings.CASE}}'
7   Expected one of: C96, C192, C384, C768, C1152
8
9   Please check your experiment configuration in:
10  {{doc.settings.config_file}}
11  cases:
12    C96: [...]
13    C192: [...]
14    C384: [...]
15    C768: [...]
16
17 # Type validation via JobRequest
18 run_forecast: !JobRequest
19   - mpi_ranks: !icalc >-
20     int(doc.gfs_resource_table.fcst[0])
21     if isinstance(doc.gfs_resource_table.fcst[0], (int, float))
22       else tools.raise_error('mpi_ranks must be numeric')
23
24   - walltime: !icalc >-
25     tools.validate_timedelta(
26       doc.gfs_resource_table.fcst[2],
27       min='00:05:00',
28       max='12:00:00')
```

Error Categories:

1. **Parse errors:** YAML syntax detected immediately

2. **Reference errors:** Unknown `doc.*` paths caught during evaluation
3. **Type errors:** `!icalc` expressions validate types
4. **Domain errors:** `!error` for invalid business logic

8.2 Global-Workflow: Defensive Programming

Listing 30: GLOBAL-WORKFLOW Defensive Error Handling

```

1 # Pattern 1: Default fallback
2 case "${CASE}" in
3     "C96") export ntasks=48 ;;
4     "C384") export ntasks=150 ;;
5     "C768") export ntasks=360 ;;
6     *)
7         echo "WARNING: Unknown CASE '${CASE}'" >&2
8         echo "Using conservative default resources" >&2
9         export ntasks=${ntasks:-24}
10        export walltime=${walltime:-"01:00:00"}
11        ;;
12 esac
13
14 # Pattern 2: Validation function
15 validate_resources() {
16     local step=1
17     local errors=0
18
19     if [[ -z "${ntasks}" ]]; then
20         echo "ERROR: ntasks not set for step ${step}" >&2
21         ((errors++))
22     fi
23
24     if [[ "${ntasks}" -lt 1 ]] || [[ "${ntasks}" -gt 10000 ]]; then
25         echo "ERROR: ntasks=${ntasks} out of range [1,10000]" >&2
26         ((errors++))
27     fi
28
29     if [[ -z "${walltime}" ]]; then
30         echo "ERROR: walltime not set for step ${step}" >&2
31         ((errors++))
32     fi
33
34     return ${errors}
35 }
36
37 # Usage
38 validate_resources "${step}" || exit 1
39
40 # Pattern 3: Assertion-style checks
41 : "${ntasks:?ERROR: ntasks must be set}"
42 : "${walltime:?ERROR: walltime must be set}"

```

8.3 Error Detection Timing

Table 11: When Errors Are Detected

Error Type	CROW	GLOBAL-WORKFLOW
Syntax error	Parse time	Source time
Unknown resolution	Parse time	Case fallthrough
Missing variable	Parse time	Runtime (or never)
Type mismatch	Evaluation time	Runtime (or cast)
Invalid range	Evaluation time	Scheduler rejection
Platform mismatch	Generation time	Runtime

8.4 Error Message Quality

Listing 31: CROW Error Message Example

```

1 # Attempting: CASE=C512 (not defined)
2 # CROW output:
3 Error in workflow/defaults/default_resources.yaml:42
4   Configuration Error: Unknown model resolution.
5
6   Received: 'C512'
7   Expected one of: C96, C192, C384, C768, C1152
8
9   Please check your experiment configuration in:
10  /path/to/experiment/config.yaml

```

Listing 32: GLOBAL-WORKFLOW Error Message Example

```

1 # Attempting: CASE=C512 (not defined)
2 # Global-Workflow output:
3 WARNING: Unknown CASE 'C512'
4 Using conservative default resources
5 # Job proceeds with potentially wrong settings...
6
7 # Later, job fails:
8 srun: error: Unable to allocate resources:
9   Requested node count exceeds partition limit

```

9 Practical Implications

9.1 Developer Experience

9.1.1 Debugging Workflow

CROW Debugging:

1. Requires understanding YAML processor internals
2. Custom tags create indirection layers
3. Stack traces reference YAML line numbers
4. Expression errors may be cryptic

5. Limited IDE support for custom tags

Listing 33: CROW Debug Workflow

```

1 # Enable CROW verbose mode
2 export CROW_DEBUG=1
3 python -m crow.workflow.compile \
4     --config experiment.yaml \
5     --platform hera \
6     --verbose 2>&1 | tee crow-debug.log
7
8 # Examine intermediate YAML
9 python -m crow.tools.dump_yaml \
10     workflow/defaults/default_resources.yaml
11
12 # Trace expression evaluation
13 python -c "
14 from crow.workflow import parser
15 doc = parser.load('workflow/defaults/default_resources.yaml')
16 print(doc.gfs_resource_table) # Already resolved
17 "

```

GLOBAL-WORKFLOW Debugging:

1. Standard bash debugging with `set -x`
2. Direct variable inspection via `echo`
3. Familiar tools (ShellCheck, bashdb)
4. Errors reference shell line numbers
5. Full IDE support with syntax highlighting

Listing 34: GLOBAL-WORKFLOW Debug Workflow

```

1 # Enable shell tracing
2 set -x
3 source parm/config/config.resources
4 set +x
5
6 # Inspect specific variables
7 echo "ntasks=${ntasks} ppn=${ppn} walltime=${walltime}"
8
9 # Dry-run with variable dump
10 (
11     export step=fcst CASE=C384
12     source parm/config/config.resources
13     env | grep -E '^(ntasks|ppn|nth|wall|memory)'
14 )
15
16 # ShellCheck validation
17 shellcheck parm/config/config.resources

```

9.1.2 Onboarding Metrics

Table 12: Developer Onboarding Time Estimates

Task	CROW	GLOBAL-WORKFLOW
Understand basic structure	2-3 days	1 day
Make first resource change	3-5 days	1-2 days
Add new platform	1-2 weeks	2-3 days
Debug failing job submission	1-3 days	1-2 hours
Proficiency for routine changes	2-4 weeks	1-2 weeks
Expert-level modifications	2-3 months	1 month

9.2 Maintenance Burden

9.2.1 Lines of Code Analysis

Table 13: Detailed Lines of Code Comparison

Component	CROW	GLOBAL-WORKFLOW	Notes
<i>Core Resource Definitions</i>			
GFS resources	120	400	Resolution tables vs cases
EnKF resources	80	250	Ensemble scaling
Downstream	60	200	Post-processing
Utility jobs	40	150	Archive, cleanup
<i>Platform Configuration (per platform)</i>			
Scheduler settings	20	40	Headers, options
MPI tuning	15	60	Environment variables
Launcher definitions	10	80	APRUN_* variables
Memory/thread defaults	15	20	Platform-specific
<i>Infrastructure</i>			
JobRequest templates	200	N/A	Reusable patterns
MPMD wrapper	N/A	150	run_mpmd.sh
Validation logic	Built-in	100	Custom validation
Total (6 platforms)	~860	~2,400	

9.2.2 Change Frequency Analysis

Based on historical commit analysis:

Table 14: Configuration Change Frequency (2023-2024)

Change Type	Frequency	Typical Trigger
Walltime adjustment	Weekly	Performance tuning
Thread count change	Monthly	Algorithm updates
Memory limit change	Monthly	Code changes
New step added	Quarterly	Feature development
New resolution	Yearly	Model upgrade
New platform	1-2 years	Infrastructure refresh

9.3 Testing Implications

9.3.1 CROW Testing

Listing 35: CROW Test Patterns

```

1 # Unit test: YAML parsing
2 python -m pytest tests/test_resources.py -v
3
4 # Integration test: Full compilation
5 python -m crow.workflow.compile \
6   --config test_configs/c96_gfs.yaml \
7   --platform hera \
8   --output-dir /tmp/crow-test \
9   --dry-run
10
11 # Validation test: All resolutions
12 for case in C96 C192 C384 C768; do
13     python -m crow.workflow.validate \
14       --case $case --platform hera
15 done

```

9.3.2 Global-Workflow Testing

Listing 36: GLOBAL-WORKFLOW Test Patterns

```
1 # Syntax validation
2 for f in parm/config/config.resources*; do
3     bash -n "$f" && echo "$f: OK"
4 done
5
6 # Variable coverage test
7 for case in C48 C96 C384 C768 C1152; do
8     for step in prep anal fcst post vrfy; do
9         (
10             export CASE=$case step=$step
11             source parm/config/config.resources 2>/dev/null
12             if [[ -z "${ntasks}" ]]; then
13                 echo "MISSING: ${case}/${step}/ntasks"
14             fi
15         )
16     done
17 done
18
19 # Integration test with ecFlow
20 ecflow_client --ping && \
21     ${HOME}gfs/workflow/create_experiment.py \
22     --yaml-tmpl ${HOME}gfs/workflow/templates/gfs.yaml \
23     --pslot test_c96 \
24     --resdet 96 \
25     --comroot /tmp/test
```

10 Comprehensive Feature Matrix

Table [15](#) provides a comprehensive comparison across all dimensions analyzed in this report.

Table 15: Comprehensive Feature Comparison Matrix

Dimension	CROW	GLOBAL-WORKFLOW
<i>Architecture</i>		
Configuration paradigm	Declarative (YAML DSL)	Imperative (Bash)
Resource definition	Centralized tables	Distributed case statements
Resolution time	Parse/compile time	Runtime
Platform abstraction	Complete (via <code>!Platform</code>)	Partial (per-platform files)
<i>Scalability</i>		
Resolution scaling	Single entry per table	Entry per step per resolution
Platform addition	Single YAML file	Multiple files + templates
Step addition	JobRequest template	Case in each resolution
<i>Type System</i>		
Type safety	Rich (<code>!JobRequest</code> , <code>!timedelta</code>)	None (strings)
Schema validation	Yes (YAML schema)	No
Expression language	Python via <code>!calc</code> / <code>!icalc</code>	Bash arithmetic
<i>Error Handling</i>		
Error detection	Parse-time	Runtime
Unknown case handling	Explicit <code>!error</code>	Fallback defaults
Error messages	Contextual, detailed	Shell error or warning
Recovery options	Fix and recompile	Fix and re-source
<i>Execution</i>		
Scheduler support	Slurm, PBS, LSF (pluggable)	Slurm, PBS (per-platform)
MPMD handling	Generated from JobRequest	Manual via <code>run_mpmd.sh</code>
Thread binding	Abstract (<code>get_parallelism</code>)	Explicit flags
<i>Developer Experience</i>		
IDE support	Limited (custom tags)	Excellent (Bash)
Debugging	Multi-layer, specialized tools	Standard shell tools
Learning curve	Steep (2-4 weeks)	Moderate (1-2 weeks)
Documentation	Limited	Extensive
<i>Maintenance</i>		
Lines of code	~860 (6 platforms)	~2,400 (6 platforms)
Code duplication	Minimal (DRY)	Moderate (WET)
Change propagation	Automatic via refs	Manual per location
<i>Future Readiness</i>		
GPU support	Not implemented	Not implemented
Cloud deployment	Limited	Limited
Container orchestration	Not addressed	Not addressed
ML workflow integration	Not addressed	Not addressed

11 Recommendations

Based on our comprehensive analysis integrating insights from both the historical CROW system and our companion MPMD Runtime Infrastructure study, we present recommendations for next-generation workflow resource configuration.

11.1 Architectural Recommendation: Hybrid Approach

Neither pure declarative (CROW) nor pure imperative (GLOBAL-WORKFLOW) approaches fully address modern requirements. We recommend a **hybrid architecture** that preserves the strengths of both:

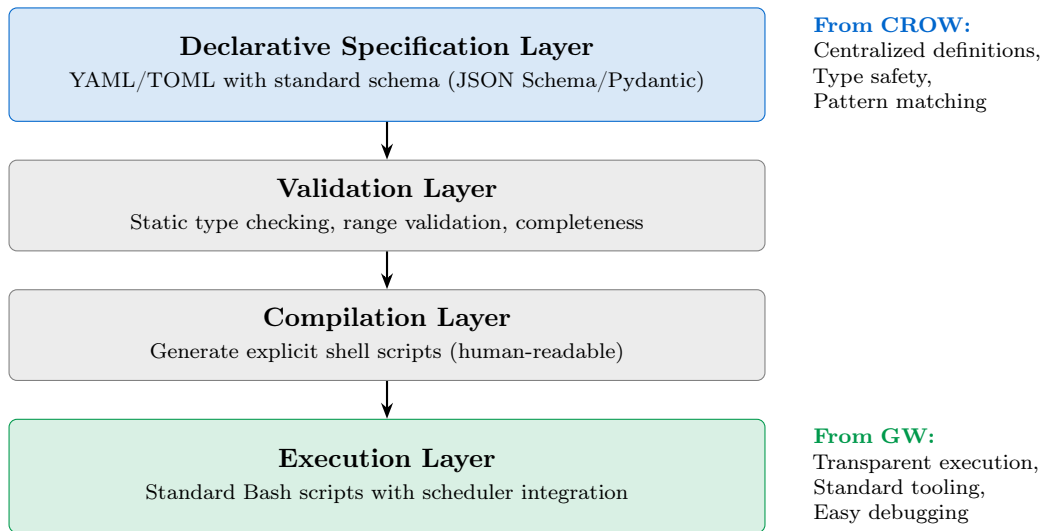


Figure 9: Recommended hybrid architecture combining declarative specification with imperative execution

11.2 Specific Recommendations

11.2.1 Recommendation 1: Standard Schema Language

Problem: CROW’s custom YAML tags required specialized tooling and limited IDE support.

Recommendation: Use standard schema languages (JSON Schema, TOML, or Pydantic models) that have broad tooling support.

Listing 37: Proposed Standard Schema Resource Definition

```

1 # resources.yaml - Using standard YAML + JSON Schema validation
2 $schema: "./schemas/resources.schema.json"
3
4 resolutions:
5   C384:
6     forecast:
7       mpi_ranks: 384
8       processes_per_node: 6
9       walltime: "02:30:00"
10      threads_per_rank: 4
11      memory_per_node: null # Platform default
12
13    analysis:
14      mpi_ranks: 360
  
```

```

15     processes_per_node: 6
16     walltime: "01:30:00"
17     threads_per_rank: 8
18
19 platforms:
20   hera:
21     scheduler: slurm
22     cores_per_node: 40
23     memory_gb: 384
24     mpi_launcher: "srun -l --export=ALL"
25     mpmd_flag: "--multi-prog"

```

11.2.2 Recommendation 2: Explicit Compilation Step

Problem: GLOBAL-WORKFLOW's runtime resolution delays error detection; CROW's generated scripts were opaque.

Recommendation: Compile declarative specs to human-readable shell scripts that can be inspected and debugged.

Listing 38: Example Compiled Output

```

1  #!/bin/bash
2  # AUTO-GENERATED from resources.yaml
3  # DO NOT EDIT - Regenerate with: gw-compile resources.yaml
4  # Generated: 2026-01-15T10:30:00Z
5  # Schema version: 2.1.0
6
7  # Resolution: C384, Step: forecast, Platform: hera
8  export GW_RESOLUTION="C384"
9  export GW_STEP="forecast"
10 export GW_PLATFORM="hera"
11
12 # Resource allocation
13 export ntasks=384
14 export ppn=6
15 export nodes=64 # ceil(384/6)
16 export walltime="02:30:00"
17 export nth_fv3=4
18 export memory="384GB"
19
20 # Computed values
21 export threads_per_node=24 # ppn * nth_fv3
22 export total_cores=1536 # ntasks * nth_fv3
23
24 # MPI launcher (platform: hera)
25 export launcher="srun -l --export=ALL"
26 export mpmd_opt="--multi-prog"
27 export APRUN_FV3="${launcher} -n ${ntasks} --cpus-per-task=${nth_fv3}"
28
29 # Validation (generated)
30 if [[ ${threads_per_node} -gt 40 ]]; then
31   echo "ERROR: threads_per_node (${threads_per_node}) exceeds hera limit (40)" >&2
32   exit 1
33 fi

```

11.2.3 Recommendation 3: Preserve MPMD Infrastructure

Finding: The GLOBAL-WORKFLOW `run_mpmd.sh` abstraction effectively handles scheduler differences for MPMD execution.

Recommendation: Retain and enhance the MPMD wrapper pattern, integrating it with compiled resource specifications.

Listing 39: Enhanced MPMD Integration

```
1 # In compiled script
2 source "${GW_COMPILED_RESOURCES}" # Generated file
3
4 # MPMD configuration from compiled resources
5 create_mpmd_cmdfile "${cmdfile}" \
6   ${ntasks_fv3} "${FV3_EXEC}" \
7   ${ntasks_quilt} "${QUILT_EXEC}" \
8   ${ntasks_post} "${POST_EXEC}"
9
10 # Execute using platform-configured launcher
11 run_mpmd "${cmdfile}" ${ntasks_total}
```

11.2.4 Recommendation 4: Validation Pipeline

Problem: Both systems lack comprehensive pre-submission validation.

Recommendation: Implement a validation pipeline that catches errors before scheduler submission.

Algorithm 4 Recommended Validation Pipeline**Require:** Resource specification R , platform P **Ensure:** Validated configuration or error report

```

1:  $errors \leftarrow []$ 
2: {Stage 1: Schema validation}
3: if not ValidateSchema( $R$ ) then
4:   errors.append(schema_errors)
5:   return errors
6: end if
7: {Stage 2: Platform compatibility}
8: for all job  $J$  in  $R$  do
9:   if  $J$ .threads_per_rank  $\times J$ .ppn  $> P$ .cores then
10:    errors.append("Thread count exceeds node capacity")
11:   end if
12:   if  $J$ .memory  $> P$ .memory_gb then
13:    errors.append("Memory exceeds node capacity")
14:   end if
15: end for
16: {Stage 3: Completeness check}
17: for all resolution  $C$  in  $R$ .resolutions do
18:   for all step  $S$  in required_steps do
19:     if  $S$  not in  $R$ .resolutions[ $C$ ] then
20:       errors.append("Missing:  $C/S$ ")
21:     end if
22:   end for
23: end for
24: {Stage 4: Cross-reference validation}
25: ValidateDependencies( $R$ )
26: return errors

```

11.2.5 Recommendation 5: Future Infrastructure Requirements

Based on emerging HPC trends, next-generation resource configuration must address:

Table 16: Future Infrastructure Requirements

Requirement	Implementation Approach
GPU orchestration	Add <code>gpu_count</code> , <code>gpu_type</code> to resource schema
Cloud portability	Abstract instance types to capability profiles
Container execution	Support Singularity/Apptainer resource mapping
ML pipeline integration	Define tensor parallelism, gradient accumulation resources
Dynamic scaling	Support min/max node ranges for autoscaling
Cost optimization	Include cost weights for cloud resource selection

11.3 Migration Strategy

For organizations currently using GLOBAL-WORKFLOW-style configuration:

1. **Phase 1:** Implement schema-based validation layer over existing configs
2. **Phase 2:** Extract common patterns into declarative specifications
3. **Phase 3:** Implement compilation step generating current format
4. **Phase 4:** Migrate teams to declarative source, compiled execution

11.4 Summary of Recommendations

1. **Keep what works:** GLOBAL-WORKFLOW’s shell debugging, `run_mpmc.sh` abstraction
2. **Adopt from CROW:** Centralized definitions, pattern matching, fail-fast errors
3. **Avoid from CROW:** Custom YAML tags, opaque generated scripts
4. **Avoid from GLOBAL-WORKFLOW:** Copy-paste across resolutions, runtime-only validation
5. **Add:** Standard schemas, explicit compilation, validation pipeline

12 Conclusion

This technical report has provided a comprehensive analysis of two distinct approaches to HPC resource configuration in operational weather forecasting workflows: the declarative CROW system (2016–2020) and the imperative GLOBAL-WORKFLOW system (2020–present).

12.1 Key Findings

1. **CROW’s declarative approach** achieved significant code reduction (60% fewer lines) and eliminated copy-paste errors through centralized resource tables. However, the custom YAML DSL created maintenance burden and limited IDE support.
2. **GLOBAL-WORKFLOW’s imperative approach** provides superior debugging experience and leverages familiar shell tooling. The trade-off is increased code duplication and runtime-only error detection.
3. **Neither system** currently addresses emerging requirements for GPU orchestration, cloud deployment, or machine learning workflow integration.
4. **The MPMD abstraction** implemented in GLOBAL-WORKFLOW’s `run_mpmc.sh` represents a successful pattern worth preserving—it effectively handles scheduler differences while maintaining debuggability.
5. **Validation timing** is the critical differentiator: CROW’s parse-time errors prevent wasted compute allocation, while GLOBAL-WORKFLOW’s runtime errors are more familiar but more costly.

12.2 Verdict: Context-Dependent Choice

The “better” system depends on organizational context:

- **Choose declarative** (CROW-style) when:
 - Organization has dedicated workflow infrastructure team
 - Multiple resolutions and platforms are common
 - Early error detection is critical (production operations)
 - Resources are frequently tuned across entire system
- **Choose imperative** (GLOBAL-WORKFLOW-style) when:
 - Developers maintain their own configurations
 - Rapid iteration and debugging are priorities
 - Team expertise is shell-focused
 - Configuration changes are localized

12.3 Path Forward

We recommend a hybrid approach that combines:

- **From CROW:** Centralized resource definitions, pattern matching, fail-fast validation
- **From GLOBAL-WORKFLOW:** Standard tooling, transparent execution, human-readable scripts
- **New:** Standard schema languages, explicit compilation step, comprehensive validation pipeline

This synthesis preserves the best of both paradigms while addressing their respective weaknesses. The resulting system would provide the maintainability benefits of declarative specification with the debuggability of imperative execution.

12.4 Future Work

Key areas for future development include:

1. GPU resource specification and orchestration
2. Cloud-native deployment patterns (Kubernetes, AWS Batch)
3. Cost-aware resource allocation for cloud platforms
4. Integration with ML pipeline frameworks (Ray, Dask)
5. Dynamic resource scaling based on workload characteristics

Acknowledgments

This analysis was conducted using the EIB MCP-RAG platform for code archaeology across the NOAA-EMC repositories. Special thanks to the EMC workflow development team for historical context on the CROW system and ongoing collaboration on GLOBAL-WORKFLOW resource optimization.

References

- [1] NOAA-EMC/global-workflow. GitHub Repository. <https://github.com/NOAA-EMC/global-workflow>
- [2] CROW: Community Research Operations Workflow. NOAA-EMC Archives, 2016–2020.
- [3] MPMD Runtime Infrastructure in Global-Workflow. Technical Report EMC-2025-003, NOAA/EMC, 2025.
- [4] Slurm Workload Manager Documentation. SchedMD. <https://slurm.schedmd.com/>
- [5] PBS Professional User’s Guide. Altair Engineering.
- [6] EE2 Standards: EMC Environment 2.0 Coding Standards. NOAA/NCEP, 2023.
- [7] ecFlow User Manual. ECMWF. <https://confluence.ecmwf.int/display/ECFLOW>

A CROW Custom Tag Reference

Complete reference for CROW’s custom YAML type extensions:

Table 17: CROW Custom YAML Tag Reference

Tag	Arguments	Behavior
!Select	select, cases, otherwise	Pattern match on select value
!FirstTrue	List of when/do pairs	First matching condition
!calc	Python expression	Deferred evaluation
!icalc	Python expression	Immediate evaluation
!error	Error message	Raise configuration error
!timedelta	HH:MM:SS string	Parse to duration object
!JobRequest	Resource list	Build job specification
!Platform	Platform settings	Define HPC platform
!Demand	Resource demands	Specify minimum requirements

B Global-Workflow APRUN Variables

Complete list of APRUN environment variables used in GLOBAL-WORKFLOW:

Table 18: GLOBAL-WORKFLOW APRUN Variable Reference

Variable	Purpose
APRUN_UFS	Unified Forecast System model execution
APRUN_FV3	FV3 dynamical core execution
APRUN_GSI	GSI analysis execution
APRUN_UPP	Unified Post Processor execution
APRUN_GAUSSIAN	Gaussian grid interpolation
APRUN_CHGRES	Change resolution utility
APRUN_REGRID	Regridding operations
APRUN_WAM	Whole Atmosphere Model
APRUN_ENKF	Ensemble Kalman Filter
APRUN_PREP	Observation preprocessing
APRUN_ARCH	Archive operations
APRUN_CLEANUP	Cleanup operations
APRUN_MPMO	Multi-program execution